

Sun'iy intellekt agentlarini yaratish

Tabiiy tilni qayta ishlash — 15-ma'ruza

A.A. Abdulali

11:00–12:20 · mavzu №15

9-iyul 2026

Prezentatsiya rejasi

- 1 Kirish: modelga fikrlab-harakat qilishni o'rgatamiz
- 2 ReAct: fikrlash va harakat sikli
- 3 Funksiya chaqirish va vositalar
- 4 Agent arxitekturasini va NLP modelni API
- 5 LangChain bilan agent qurish
- 6 Xulosa va amaliyotga ko'prik

Takrorlash: o'tgan darsdan ikki savol

1-savol

L14 da RAG tashqi bilimni qidirib javobga qo'shdi. Bu **bitta** amal edi — agar vazifa ko'p qadam talab qilsa-chi?

2-savol

Foydalanuvchi "Bu sharh ijobiymi va o'xshashlarini top" desa, LLM o'zi sentiment va qidiruvni qanday bajaradi?

Takrorlash: o'tgan darsdan ikki savol

1-savol

L14 da RAG tashqi bilimni qidirib javobga qo'shdi. Bu **bitta** amal edi — agar vazifa ko'p qadam talab qilsa-chi?

2-savol

Foydalanuvchi "Bu sharh ijobiymi va o'xshashlarini top" desa, LLM o'zi sentiment va qidiruvni qanday bajaradi?

1-javob

RAG — bitta vosita (retrieve). Ko'p qadamli vazifa uchun model **rejalashtirishi** va bir nechta vositani chaqirishi kerak.

2-javob

LLM o'zi qila olmaydi — unga **vositalar** (tools) beramiz va **fikrlab-harakat** qildiramiz: bu **agent**.

Siz bugungi dars oxirida quyidagilarni bajara olasiz:

- ➊ **Tushuntira olasiz:** ReAct (Fikrlash + Harakat) siklini va nega agent LLM dan kuchliroq.
- ➋ **Kuzata olasiz:** agent to'g'ri vositani tanlashini — query uchun `sentiment_classify` (> 0.7 ishonch).
- ➌ **Tushuntira olasiz:** funksiya chaqirish (tools) va NLP modelni API sifatida joylashtirish.
- ➍ **Tushuntira olasiz:** LangChain bilan agent qurish arxitekturasini.

L13/L14 dan: fine-tuning, RAG. P1–P14 modullari (m04, m10, m12, m13, m14). Tushuncha: prompt, JSON, funksiya imzosi.

Bugungi dars rejasi:

Tsikl	Mavzu	Vaqt	Hosil natija	bo'luvchi
1	ReAct: fikrlash + harakat	16 daq	to'g'ri tool tanlash	
2	Funksiya chaqirish va tools	16 daq	tool sxemasi	
3	Agent arxitekturasini va API	16 daq	NLP model API	
4	LangChain bilan agent qurish	16 daq	5 ta tool ulanishi	
	Xulosa va 15-amaliyotga ko'rsatmalar	10 daq	Ertaga nima qurishingiz	

Bu dars agent tizimiga olib keladi

Avvalgi modullaringiz agent **vositalariga** aylanadi. Bugungi "to'g'ri tool tanlash" ertangi P15 da assert bo'ladi.

Bugungi dars uchun muammo: LLM o'zi harakat qila olmaydi

Faqat LLM qayerda qiynaladi:

Tashqi dunyoga ta'sir yo'q

LLM matn yaratadi, ammo qidiruv, hisob-kitob yoki API chaqiruvni o'zi bajara olmaydi.

Ko'p qadamli reja yo'q

"Sharhni tahlil qil va o'xshashini top" — bu bir nechta qadam; LLM ularni o'zi tartiblab bajara olmaydi.

Bugungi yechim:

- 1 Modelga **vositalar** (tools) beramiz (sentiment, qidiruv, ...).
- 2 Model **fikrlaydi** (qaysi vosita kerak) va **harakat** qiladi (chaqiradi).
- 3 Natijani **kuzatadi** va kerak bo'lsa takrorlaydi — **agent**.

ReAct (Yao, 2022): Reasoning + Acting.
Avvalgi modullaringiz agent vositalariga aylanadi.

Prezentatsiya rejasi

- 1 Kirish: modelga fikrlab-harakat qilishni o'rgatamiz
- 2 **ReAct: fikrlash va harakat sikli**
- 3 Funksiya chaqirish va vositalar
- 4 Agent arxitekturasini va NLP modelni API
- 5 LangChain bilan agent qurish
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: o'ylab, harakat qilib, natijaga qarab davom etamiz

Inson murakkab vazifani shunday yechadi: **o'ylaydi** (nima kerak), **harakat** qiladi (biror amal), **natijaga qaraydi**, keyin davom etadi.

ReAct shu siklni modellaydi: **Thought** (fikir) → **Action** (vosita chaqiruvi) → **Observation** (natija), kerak bo'lguncha takror.

Asosiy ark: L14 RAG — bitta amal (retrieve). Agent esa ko'p vositani **re-jalashtirib**, ketma-ket chaqiradi.

$$a_t = \text{LLM}(q, h_{t-1}), \quad o_t = \text{tool}(a_t), \quad h_t = h_{t-1} \cup \{a_t, o_t\}$$

bunda: q — foydalanuvchi so'rovi; h_{t-1} — oldingi qadamlar tarixi (Thought/Action/Observation); a_t — tanlangan harakat (Thought + tool chaqiruvi); o_t — vosita natijasi (Observation). Sikl yakuniy javob ("Final Answer") chiqarilguncha takrorlanadi. LLM bu yerda **rejalashtiruvchi** (qaysi vosita), tool esa ijrochi.

Keltirib chiqarish: nega fikr va harakatni qo'shamiz?

1-qadam. Faqat fikrlash (Chain-of-Thought): model o'ylaydi, ammo tashqi fakti **tekshira olmaydi** — gallyutsinatsiya xavfi.

Keltirib chiqarish: nega fikr va harakatni qo'shamiz?

1-qadam. Faqat fikrlash (Chain-of-Thought): model o'ylaydi, ammo tashqi fakti **tekshira olmaydi** — gallyutsinatsiya xavfi. **2-qadam.** Faqat harakat: model vositani chaqiradi, ammo **nega** ekanini rejalashtirmaydi — noto'g'ri tartib.

Keltirib chiqarish: nega fikr va harakatni qo'shamiz?

1-qadam. Faqat fikrlash (Chain-of-Thought): model o'ylaydi, ammo tashqi fakti **tekshira olmaydi** — gallyutsinatsiya xavfi. **2-qadam.** Faqat harakat: model vositani chaqiradi, ammo **nega** ekanini rejalashtirmaydi — noto'g'ri tartib. **3-qadam.** ReAct ikkisini birlashtiradi: har qadam avval **Thought** (reja), keyin **Action** (amal).

Keltirib chiqarish: nega fikr va harakatni qo'shamiz?

1-qadam. Faqat fikrlash (Chain-of-Thought): model o'ylaydi, ammo tashqi fakti **tekshira olmaydi** — gallyutsinatsiya xavfi. **2-qadam.** Faqat harakat: model vositani chaqiradi, ammo **nega** ekanini rejalashtirmaydi — noto'g'ri tartib. **3-qadam.** ReAct ikkisini birlashtiradi: har qadam avval **Thought** (reja), keyin **Action** (amal). **4-qadam.** **Observation** natijani qaytaradi $\Rightarrow$ keyingi fikr unga tayanadi (gallyutsinatsiya kamayadi, reja aniqlashadi).

Xulosa: fikr rejani, harakat fakti beradi. Endi bitta ReAct qadamini qo'lda kuzatamiz.

Qo'lda misol: agent to'g'ri vositani tanlaydi

```
query = "Uzum Market ilovasi yaxshimi?"; Tools = [sentiment_classify, retrieve_docs, summarize].
```

ReAct 1-qadam:

Thought: "Bu — hissiyot savoli → sentiment_classify kerak."

Action: sentiment_classify(text=query)

Observation: {sentiment: ijobiy, confidence: 0.82}

Agent summarize ni emas,
sentiment_classify ni tan-
ladi; ishonch $0.82 > 0.7$.

Ertangi P15 birinchi assert:

```
assert tool ==  
    "sentiment_classify"  
assert conf > 0.7  
# Ma'ruza L15 [11]-slayd
```

Sizning vazifangiz: boshqa so'rov uchun vosita

Savol — 60 soniya

query = "Bu hujjatni qisqacha bayon qil." Tools = [sentiment_classify, retrieve_docs, summarize].

Agent qaysi vositani tanlaydi?

Sizning vazifangiz: boshqa so'rov uchun vosita

Savol — 60 soniya

query = "Bu hujjatni qisqacha bayon qil." Tools = [sentiment_classify, retrieve_docs, summarize].

Agent qaysi vositani tanlaydi?

Javob

summarize — "qisqacha bayon" xulosalashni bildiradi.

Agent so'rov **maqsadiga** qarab vositani tanlaydi: hissiyot → sentiment; qidiruv → retrieve; xulosa → summarize.

Kod $\leftrightarrow$ formula: ReAct sikli

```
1 def react(query, tools, llm, max_steps=5):
2     history = []
3     for _ in range(max_steps):
4         step = llm.plan(query, history)    #
5         Thought + Action
6         if step.is_final:
7             return step.answer
8         obs = tools[step.action](step.args) #
9         Observation
10        history.append((step, obs))
11    return llm.finalize(history)
```

Kod $\rightarrow$ formula:

Kod	Formula
llm.plan	$a_t = \text{LLM}(q, h)$
tools[...]	$o_t = \text{tool}(a_t)$
history	h_t

P15 da m15 shu siklni LangChain bilan quradi.

ReAct tuzog'i: cheksiz sikl

Sikl tugamasligi

Agent yakuniy javobga kelolmay, vositalarni qayta-qayta chaqirishi mumkin (cheksiz sikl) — vaqt va narx isrof bo'ladi.

Yechim: `max_steps` chegarasi; aniq tool tavsiflari; "Final Answer" formatini majburlash.

Prezentatsiya rejasi

- 1 Kirish: modelga fikrlab-harakat qilishni o'rgatamiz
- 2 ReAct: fikrlash va harakat sikli
- 3 Funksiya chaqirish va vositalar**
- 4 Agent arxitekturasini va NLP modelni API
- 5 LangChain bilan agent qurish
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: modelga "asboblarni qutisi" beramiz

Har vosita — nomi, tavsifi va argumentlari bo'lgan funksiya. Modelga ularning **ro'yxati** beriladi.

Model so'rovni ko'rib, qaysi vositani qanday argument bilan chaqirishni **tanlaydi** — bu "funksiya chaqirish" (function calling).

Tavsif (description) muhim — model aynan shunga qarab vositani tanlaydi.
Yomon tavsif $\Rightarrow$ noto'g'ri tanlov.

Tool va funksiya chaqirish: rasmiy ta'rif

$$\text{tool} = (\text{name}, \text{description}, \text{schema}), \quad (\text{name}^*, \text{args}^*) = \text{LLM}(q, \{\text{tool}_i\})$$

bunda: name — vosita nomi; description — u nima qilishi (model shunga qarab tanlaydi); schema — argumentlar turi; $(\text{name}^*, \text{args}^*)$ — model tanlagan vosita va argumentlar. Model matn emas, **strukturali** chaqiruv qaytaradi (JSON): qaysi funksiya, qanday argument bilan.

Keltirib chiqarish: model funksiyani qanday tanlaydi?

1-qadam. Har vositaning **tavsifi** promptga qo'shiladi (nima qiladi, qachon ishlatiladi).

Keltirib chiqarish: model funksiyani qanday tanlaydi?

1-qadam. Har vositaning **tavsifi** promptga qo'shiladi (nima qiladi, qachon ishlatiladi).

2-qadam. Model so'rov ma'nosini tavsiflar bilan **solishtiradi** — eng mosini tanlaydi.

Keltirib chiqarish: model funksiyani qanday tanlaydi?

1-qadam. Har vositaning **tavsifi** promptga qo'shiladi (nima qiladi, qachon ishlatiladi).

2-qadam. Model so'rov ma'nosini tavsiflar bilan **solishtiradi** — eng mosini tanlaydi.

3-qadam. Argumentlarni so'rovdan ajratib, **schema** bo'yicha to'ldiradi (masalan text=query).

Keltirib chiqarish: model funksiyani qanday tanlaydi?

1-qadam. Har vositaning **tavsifi** promptga qo'shiladi (nima qiladi, qachon ishlatiladi).

2-qadam. Model so'rov ma'nosini tavsiflar bilan **solishtiradi** — eng mosini tanlaydi.

3-qadam. Argumentlarni so'rovdan ajratib, **schema** bo'yicha to'ldiradi (masalan text=query). **4-qadam.** Strukturali chaqiruv (JSON) qaytariladi $\Rightarrow$ tizim funksiyani ishonchli ishga tushiradi.

Xulosa: aniq tavsif + schema = ishonchli tanlov. Endi tool tanlashni qo'lda ko'ramiz.

Qo'lda misol: tavsifga qarab tool tanlash

`spell_correct`: imlo xatosini tuzatadi. `extract_entities`: nom/joy/tashkilotni ajratadi.

`query` = "Bu matndagi shahar nomlarini top."

Tavsiflarni solishtiramiz: "shahar nomlari" → joy →
`extract_entities` (nom/joy ajratadi).

Tanlov: `extract_entities`,
`args={text: query}`. Tavsif
mosligi tanlovni belgiladi.

Sizning vazifangiz: qaysi vosita?

Savol — 60 soniya

```
query = "Bu so'rovda imlo xatosi bormi: 'oqua'?"    Tools = [spell_correct,
extract_entities].
```

Qaysi vosita va qanday argument?

Sizning vazifangiz: qaysi vosita?

Savol — 60 soniya

query = "Bu so'rovda imlo xatosi bormi: 'oqua'?" Tools = [spell_correct, extract_entities].

Qaysi vosita va qanday argument?

Javob

spell_correct, args={word: "oqua"} — "imlo xatosi" tavsifga mos.

Model maqsad ("imlo") ni tavsif ("imlo tuzatadi") bilan bog'laydi va argumentni so'rovdan oladi.

Kod $\leftrightarrow$ formula: tool ta'rifi

```
1 tools = [  
2   {  
3     "name": "sentiment_classify",  
4     "description": "Matn hissiyotini aniqlaydi (  
5       ijobiy/salbiy)",  
6     "args": {"text": "str"},  
7   },  
8   # ... retrieve_docs, summarize, ...  
9 ]
```

Kod $\rightarrow$ formula:

Kod	Formula
name	name
description	description
args	schema

Tavsif aniq bo'lsa, model to'g'ri tanlaydi. P15 da m13/m14/... shu formatda o'raladi.

Tool tuzog'i: noaniq tavsif — noto'g'ri tanlov

Yomon description

Tavsif noaniq yoki o'xshash bo'lsa, model noto'g'ri vositani tanlaydi (masalan `summarize` o'rniga `retrieve_docs`).

Yechim: har vosita uchun aniq, farqlanadigan tavsif va argument sxemasi; misol bilan.

Prezentatsiya rejasi

- 1 Kirish: modelga fikrlab-harakat qilishni o'rgatamiz
- 2 ReAct: fikrlash va harakat sikli
- 3 Funksiya chaqirish va vositalar
- 4 Agent arxitekturasini va NLP modelni API**
- 5 LangChain bilan agent qurish
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: modellaringizni xizmat sifatida ulaymiz

Agent vositalari ko'pincha **tashqi xizmatlar** bo'ladi: sizning NLP modelingiz (m13) HTTP API sifatida ishlaydi, agent uni chaqiradi.

Model bir marta **API** ortiga joylashtiriladi (deploy), so'ng agent yoki boshqa ilova undan foydalanadi.

FastAPI bilan model HTTP endpoint bo'ladi: `POST /predict`. Agent tool shu endpointga so'rov yuboradi.

NLP modelni API sifatida: rasmiy ta'rif

POST /predict : request = {text} $\longrightarrow$ response = {sentiment, confidence}

bunda: request — kiruvchi JSON (matn); response — chiquvchi JSON (sinf + ishonch); endpoint — model joylashgan manzil (URL); agent tool shu shartnoma bo'yicha chaqiradi. Model logikasi bir joyda; ko'p iste'molchi (agent, veb, mobil) bir xil API ni ishlatadi.

Keltirib chiqarish: nega modelni API qilamiz?

1-qadam. Model og'ir (vaznlar, kutubxonalar) — har ilovaga nusxa qilish isrof.

Keltirib chiqarish: nega modelni API qilamiz?

1-qadam. Model og'ir (vaznlar, kutubxonalar) — har ilovaga nusxa qilish isrof. **2-qadam.** Modelni bir marta **xizmat** (server) qilamiz, u so'rovlarni qabul qiladi.

Keltirib chiqarish: nega modelni API qilamiz?

1-qadam. Model og'ir (vaznlar, kutubxonalar) — har ilovaga nusxa qilish isrof. **2-qadam.** Modelni bir marta **xizmat** (server) qilamiz, u so'rovlarni qabul qiladi. **3-qadam.** Iste'molchilar (agent, veb) faqat **JSON so'rov** yuboradi — modelni bilishi shart emas.

Keltirib chiqarish: nega modelni API qilamiz?

1-qadam. Model og'ir (vaznlar, kutubxonalar) — har ilovaga nusxa qilish isrof. **2-qadam.** Modelni bir marta **xizmat** (server) qilamiz, u so'rovlarni qabul qiladi. **3-qadam.** Iste'molchilar (agent, veb) faqat **JSON so'rov** yuboradi — modelni bilishi shart emas. **4-qadam.** Markazlashgan: yangilash, monitoring, masshtablash bir joyda (L16 MLOps).

Xulosa: API — modelni ko'p iste'molchiga ulashning standart yo'li. Endi so'rov-javobni qo'lda ko'ramiz.

Qo'lda misol: API so'rov va javob

```
Request: {"text": "mahsulot zo'r"}
```

Javob:

```
{"sentiment": "ijobiy",  
  "confidence": 0.91}
```

Agent tool shu javobni Observation sifatida oladi.

Bir endpoint — ko'p iste'molchi.
JSON shartnoma o'zgarmasa, agent
ishlayveradi.

Sizning vazifangiz: javob JSON ini yozing

Savol — 60 soniya

POST /predict ga {"text": "juda yomon xizmat"} yuborildi. Model salbiy, ishonch 0.88.

Javob JSON i qanday bo'ladi?

Sizning vazifangiz: javob JSON ini yozing

Savol — 60 soniya

POST /predict ga {"text": "juda yomon xizmat"} yuborildi. Model salbiy, ishonch 0.88.

Javob JSON i qanday bo'ladi?

Javob

```
{"sentiment": "salbiy", "confidence": 0.88}
```

Shartnoma bir xil: sentiment + confidence. Yorliqlar ijobiy/salbiy (kurs bo'yicha qulflangan).

Kod ↔ formula: FastAPI endpoint

```
1 from fastapi import FastAPI
2 app = FastAPI()
3 clf = FineTunedClassifier(); clf.load("m13")
4
5 @app.post("/predict")
6 def predict(body: dict):
7     text = body["text"]
8     return {"sentiment": clf.predict(text),
9           "confidence": max(clf.predict_proba(
10                           text).values())}
```

Kod → formula:

Kod	Formula
body	request
return	response
/predict	endpoint

Bu — M4 (P16) FastAPI ishi bilan bir xil; agent tool shu API ni chaqiradi.

API tuzog'i: shartnoma o'zgarsa, agent buziladi

Mos kelmagan JSON

API javob formati o'zgarsa (masalan confidence olib tashlansa), agent tool natijani noto'g'ri o'qiydi — butun zanjir buziladi.

Yechim: barqaror JSON shartnoma; versiyalash; sxema tekshiruvi (validation).

Prezentatsiya rejasi

- 1 Kirish: modelga fikrlab-harakat qilishni o'rgatamiz
- 2 ReAct: fikrlash va harakat sikli
- 3 Funksiya chaqirish va vositalar
- 4 Agent arxitekturasini va NLP modelni API
- 5 LangChain bilan agent qurish**
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: agent qismlarini birlashtiruvchi ramka

ReAct siklini, tool'larni va prompt'ni qo'lda yozish murakkab. **LangChain** — ularni standart qismlar sifatida beradi: Tool, AgentExecutor, ReAct prompt.

Siz vositalarni e'lon qilasiz, LangChain ReAct siklini boshqaradi.

Bizning modullar (m13, m14, m12, m04, m10) Tool obyektlariga o'raladi — agent ularni chaqiradi.

LangChain agent: rasmiy ta'rif

$$\text{agent} = \text{AgentExecutor}(\text{LLM}, \{\text{Tool}_i\}, \text{prompt}), \quad \hat{y} = \text{agent.run}(q)$$

bunda: LLM — rejalashtiruvchi model; $\{\text{Tool}_i\}$ — o'ralgan vositalar (m13/m14/...); prompt — ReAct shabloni; agent.run — so'rovni ReAct sikli orqali yakuniy javobgacha olib boradi. LangChain Thought/Action/Observation tahlilini va sikl boshqaruvini o'z zimmasiga oladi.

Keltirib chiqarish: nega ramka (LangChain)?

1-qadam. ReAct prompt, javob tahlili (parsing), sikl boshqaruvi — takrorlanuvchi va xatoga moyil.

Keltirib chiqarish: nega ramka (LangChain)?

1-qadam. ReAct prompt, javob tahlili (parsing), sikl boshqaruvi — takrorlanuvchi va xatoga moyil. **2-qadam.** LangChain bularni **standartlashtiradi**: Tool obyekti, ReAct prompt shabloni.

Keltirib chiqarish: nega ramka (LangChain)?

1-qadam. ReAct prompt, javob tahlili (parsing), sikl boshqaruvi — takrorlanuvchi va xatoga moyil. **2-qadam.** LangChain bularni **standartlashtiradi**: Tool obyekti, ReAct prompt shabloni. **3-qadam.** Siz faqat vositalarni e'lon qilasiz (nom, tavsif, funksiya); qolganini AgentExecutor bajaradi.

Keltirib chiqarish: nega ramka (LangChain)?

1-qadam. ReAct prompt, javob tahlili (parsing), sikl boshqaruvi — takrorlanuvchi va xatoga moyil. **2-qadam.** LangChain bularni **standartlashtiradi**: Tool obykti, ReAct prompt shabloni. **3-qadam.** Siz faqat vositalarni e'lon qilasiz (nom, tavsif, funksiya); qolganini AgentExecutor bajaradi. **4-qadam.** Modullaringiz (m13/m14/...) tez ulanadi $\Rightarrow$ ishlovchi agent kam kod bilan.

Xulosa: LangChain — agent "armaturasi". Endi tool ulanishini sanab ko'ramiz.

Qo'lda misol: hujjat yordamchisi uchun 5 vosita

```
m13 → sentiment_classify;    m14 → retrieve_docs;    m12 → summarize;    m04 →  
spell_correct; m10 → extract_entities.
```

Hisob: 5 ta modul → **5 ta** Tool obyekti → bitta
AgentExecutor.

Butun kursdagi ish bitta agentga jam-
lanadi — "O'zbek hujjat yordamchisi".

Sizning vazifangiz: yangi vosita qo'shish

Savol — 60 soniya

Agentga tarjima imkonini qo'shmoqchisiz (m11 Seq2SeqTranslator). Nechta Tool obyekti bo'ladi va nima e'lon qilasiz?

Sizning vazifangiz: yangi vosita qo'shish

Savol — 60 soniya

Agentga tarjima imkonini qo'shmoqchisiz (m11 Seq2SeqTranslator). Nechta Tool obyekti bo'ladi va nima e'lon qilasiz?

Javob

6 ta tool. Yangi Tool: name = translate, description = "matnni tarjima qiladi", funksiya = m11.translate.

Agentni kengaytirish = yangi Tool qo'shish — ReAct sikli o'zgarmaydi.

Kod $\leftrightarrow$ formula: LangChain agent

```
1 from langchain.agents import Tool, AgentExecutor
2
3 tools = [
4     Tool("sentiment_classify", m13.predict,
5         "Matn hissiyotini aniqlaydi"),
6     Tool("retrieve_docs", m14.answer,
7         "Bilim bazasidan qidiradi"),
8 ]
9 agent = AgentExecutor.from_agent_and_tools(llm,
10     tools)
11 print(agent.run("Bu sharh ijobiymi?"))
```

Kod $\rightarrow$ formula:

Kod	Formula
Tool	$Tool_i$
AgentExec. .run(q)	agent $\hat{y}$

P15 da m15 aynan shu tariqa 5 modulni birlashtiradi.

LangChain tuzog'i: ko'p tool — chalkash tanlov

Juda ko'p yoki o'xshash tool

Vositalar soni ortsa yoki tavsiflari o'xshash bo'lsa, agent tanlovda adashadi va noto'g'ri zanjir quradi.

Yechim: kam, aniq farqlanadigan vositalar; har biriga ravshan tavsif va misol; kerak bo'lsa guruhlash.

Prezentatsiya rejasi

- 1 Kirish: modelga fikrlab-harakat qilishni o'rgatamiz
- 2 ReAct: fikrlash va harakat sikli
- 3 Funksiya chaqirish va vositalar
- 4 Agent arxitekturasini va NLP modelni API
- 5 LangChain bilan agent qurish
- 6 Xulosa va amaliyotga ko'prik

O'zbek tilida: agent fikrlashi va tool chaqiruvi

1. O'zbekcha fikrlash:

ReAct "Thought" o'zbek tilida bo'lsa, LLM mantiqi qanchalik ishonchli? Ko'p modellar inglizchada kuchliroq fikrlaydi.

Xavf

O'zbekcha murakkab so'rovda noto'g'ri tool tanlash ehtimoli ortishi mumkin.

2. Yechim va sinov:

Tool tavsiflarini aniq yozish; kerak bo'lsa ichki fikrlashni inglizchada, yakuniy javobni o'zbekchada.

O'zbekcha so'rovlarda tool tanlashni sinab ko'rish (test) shart — kam-resursli til uchun ishonchlilik o'lchanadi.

Sintez: oddiy LLM, RAG va agent

Jihat	RAG (L14)	Agent (L15)
Amallar soni	bitta (retrieve)	ko'p (rejalashtiriladi)
Qaror	yo'q (sobit oqim)	qaysi tool, qachon (ReAct)
Vositalar	qidiruv	sentiment/qidiruv/xulosa/...
Tashqi ta'sir	faqat o'qish	API/funksiya chaqirish

Asosiy g'oya

Agent — ReAct (fikir + harakat) orqali ko'p vositani rejalashtirib chaqiradi. LLM rejalashtiruvchi, tool'lar ijrochi. Avvalgi modullaringiz agent vositalariga aylanadi.

Seminal manba: Yao va boshqalar (2022)

Yao, S., Zhao, J., Yu, D., va boshq. (2022). **ReAct: Synergizing Reasoning and Acting in Language Models.** *ICLR 2023.*

Ahamiyati:

- Fikrlash (reasoning) va harakatni (acting) bitta siklda birlashtirdi.
- Tashqi vositalar bilan gallyutsinatsiyani kamaytirdi.
- Zamonaviy LLM agentlarining (AutoGPT, LangChain) asosiga aylandi.

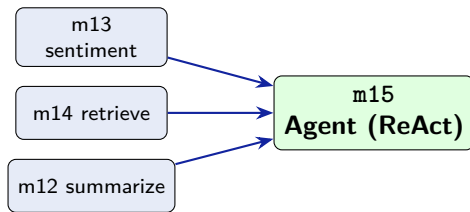
Muhokama savoli (P15 gacha o'ylang)

O'zbekcha so'rovda agent to'g'ri tool tanlashini qanday sinaysiz? "Thought" ni o'zbekchada yuritish ishonchlilikka qanday ta'sir qiladi?

Bugungi maqsadlar bajarildi

- ① ✓ **Tushuntirdik:** ReAct (Fikrlash + Harakat) sikli va uning ustunligi.
- ② ✓ **Kuzatdik:** agent to'g'ri vositani tanlashini — `sentiment_classify`, ishonch $0.82 > 0.7$.
- ③ ✓ **Tushuntirdik:** funksiya chaqirish (tools) va NLP modelni API sifatida joylashtirish.
- ④ ✓ **Tushuntirdik:** LangChain bilan agent qurish.

Ertaga P15: hujjat yordamchisi agentini quramiz



P15 da qurasiz:

- m15 DocumentAssistantAgent: ReAct + 5 tool
- m13/m14/m12/m04/m10 ni Tool qilib ulash
- o'zbekcha so'rovga yakuniy javob

Birinchi assert (bu slayd [I1]):

```
assert tool ==  
    "sentiment_classify"  
assert conf > 0.7  
# Ma'ruza L15 [I1]-slayd bilan  
# solishtiring (ReAct tool tanlash)
```

- ① Yao, S., va boshq. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023.
- ② Schick, T., va boshq. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. NeurIPS 2023.
- ③ Wei, J., va boshq. (2022). *Chain-of-Thought Prompting Elicits Reasoning in LLMs*. NeurIPS 2022.
- ④ python.langchain.com — Agents va Tools hujjatlari.
- ⑤ fastapi.tiangolo.com — API joylashtirish hujjatlari.

Ilova S1: ReAct prompt shabloni (qisqacha)

ReAct LLM ga quyidagi formatda fikrlashni so'raydi:

```
Savol: {query}
Thought: men nima qilishim kerak?
Action: {tool_name}
Action Input: {args}
Observation: {tool natijasi}
... (takrorlanadi) ...
Thought: endi javobni bilaman
Final Answer: {yakuniy javob}
```

bunda: Model shu formatga amal qiladi; tizim Action/Action Input ni o'qib vositani chaqiradi, natijani Observation ga qo'yadi.

Ilova S2: agent xotirasi (memory)

Ko'p qadamli yoki ko'p navbatli (multi-turn) suhbatda agent **xotira** saqlaydi:

- **Qisqa muddatli:** joriy ReAct tarixi (h_t).
- **Uzoq muddatli:** oldingi suhbatlar (ko'pincha vektor DB da, L14).

bunda: Xotira agentga oldingi qadamlar va suhbatlarni eslab, izchil javob berishga yordam beradi — ammo kontekst oynasi va narxni oshiradi (L14 [I2]).